

Software Architecture, Patterns & Reuse

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

6 March 2026

Today's Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns
- 3 Design Patterns
- 4 Software Reuse & Open Source
- 5 Component-Based Software Engineering
- 6 Next Steps

Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns
- 3 Design Patterns
- 4 Software Reuse & Open Source
- 5 Component-Based Software Engineering
- 6 Next Steps

Recap: System Modelling & UML

Last week, we explored how to abstract our system requirements into visual models.

- **Use Case Diagrams:** External perspective (Actors and their goals).
- **Sequence Diagrams:** Interaction perspective over time (Lifelines and Messages).
- **Class Diagrams:** Structural perspective (OOP Encapsulation, Inheritance, Aggregation vs Composition).
- **Activity Diagrams:** Behavioral perspective (Flow of control, forks, and decisions).

Today's Learning Objectives

By the end of this lecture, you should be able to:

- 1 **Differentiate** between high-level Architectural Patterns and low-level Design Patterns.
- 2 **Identify** common architectural patterns like Layered and Model-View-Controller (MVC).
- 3 **Apply** standard GoF Design Patterns (e.g., Singleton, Observer) to solve recurring coding problems.
- 4 **Evaluate** the benefits and risks of Software Reuse and Open Source licensing in modern engineering.

Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns**
- 3 Design Patterns
- 4 Software Reuse & Open Source
- 5 Component-Based Software Engineering
- 6 Next Steps

What is an Architectural Pattern?

Definition

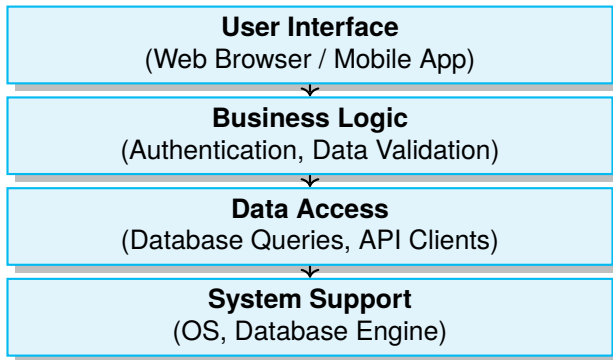
Patterns are a means of representing, sharing, and reusing knowledge about a commonly occurring problem in software architectures.

Why use them?

- They provide a stylized description of good design practice that has been tried and tested.
- Architecture heavily dictates performance, robustness, and distributability.
- It enhances stakeholder communication (e.g., "We are using a Client-Server model").

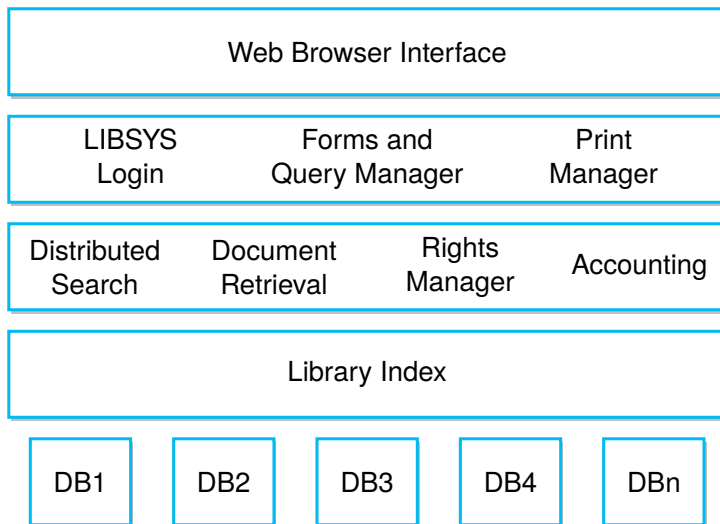
Pattern 1: Layered Architecture

Organizes the system into a set of layers, each providing a set of services to the layer above it.



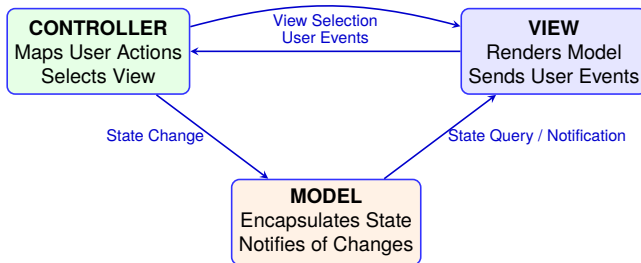
Advantage: You can replace an entire layer (e.g., swapping a web UI for a mobile UI) without affecting the core business logic.

Example: Library System Layered Architecture

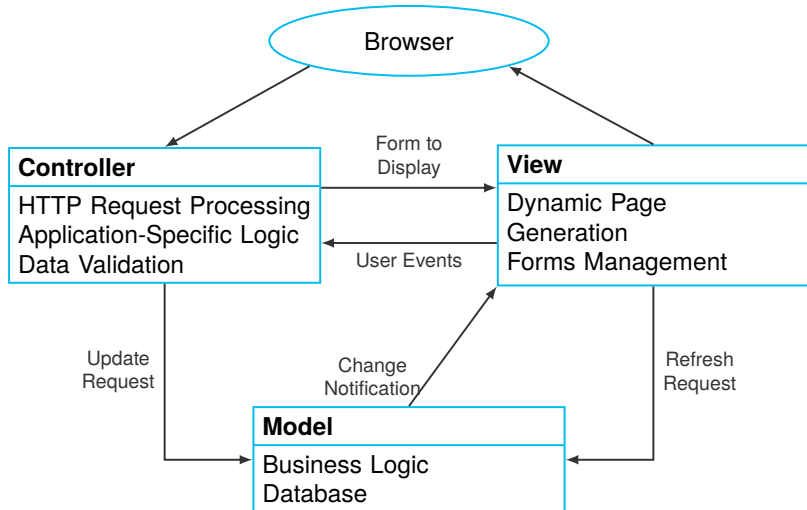


Pattern 2: Model-View-Controller (MVC)

Separates presentation and interaction from the system data.
Highly standard for Web Applications.

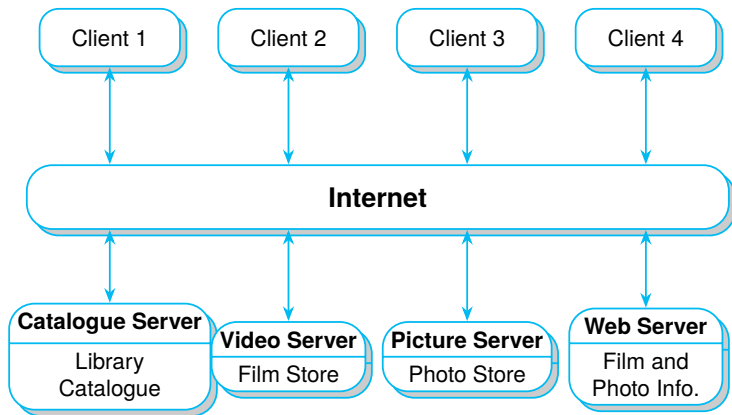


Example: Web Application Architecture (MVC)



Pattern 3: Client-Server Architecture

A distributed architecture that partitions tasks between providers of a resource or service (servers) and service requesters (clients).



Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns
- 3 Design Patterns**
- 4 Software Reuse & Open Source
- 5 Component-Based Software Engineering
- 6 Next Steps

Design Patterns

Definition

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design.

Key Characteristics:

- It is not a finished design that can be directly transformed into code.
- It is a template for how to solve a problem that can be used in many different situations.
- It describes the problem, the solution, and the consequences of using that solution.

Design Patterns

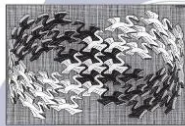
The Gang of Four (GoF)
Design Patterns refer to a set of 23 classic software design patterns, documented in the highly influential 1994 book.

The book quickly became a cornerstone of object-oriented design, and the patterns it describes are still widely used today.

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES



Available at the library.

Architecture vs. Design Patterns

- **Architectural Patterns** deal with the high-level organization of your entire software system (e.g., MVC).
- **Design Patterns** are used at a lower level to solve recurring problems at the code/class construction level.

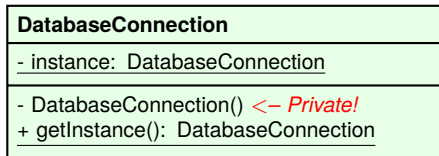
The "Gang of Four" (GoF) Categories:

- 1 **Creational:** How objects are created (e.g., Singleton, Factory).
- 2 **Structural:** How objects are composed (e.g., Facade, Adapter).
- 3 **Behavioral:** How objects communicate (e.g., Observer, Iterator).

1. Creational: The Singleton Pattern

Problem: You need exactly *one* instance of a class across the entire application (e.g., a Database Connection, or a Robot API Client).

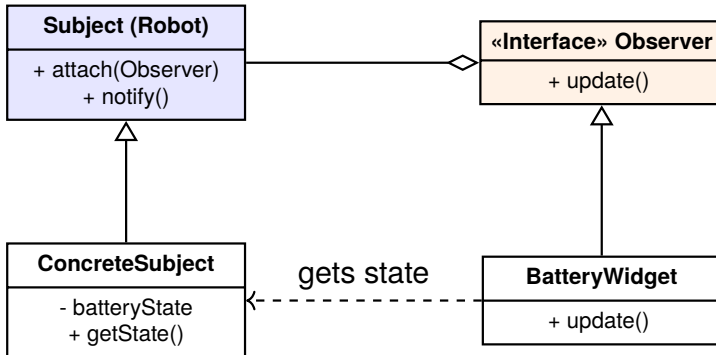
Solution: Make the constructor private. Provide a static method that creates the instance only if it doesn't exist yet, and returns the existing one if it does.



Note: The underline in UML means a property/method is Static.

2. Behavioral: The Observer Pattern

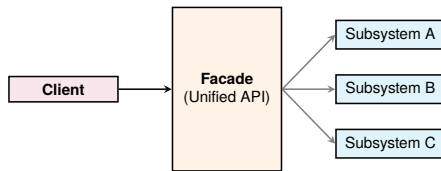
Problem: You need to tell several objects that the state of some other object has changed (e.g., updating multiple UI widgets when the Robot's battery drops).



3. Structural: The Facade Pattern

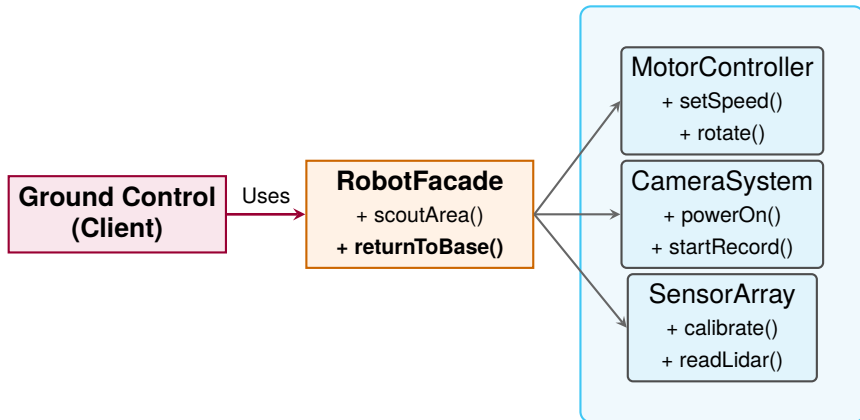
Problem: You need to "tidy up the interfaces to a number of related objects that have often been developed incrementally". Complex subsystems with many moving parts can overwhelm the client code.

Solution: Provide a unified, simplified, higher-level interface (a "Facade") that makes the subsystem easier to use. The client talks *only* to the Facade, which handles the messy internal delegation.



The Facade Pattern - Example

Complex Robot Subsystems



Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns
- 3 Design Patterns
- 4 Software Reuse & Open Source**
- 5 Component-Based Software Engineering
- 6 Next Steps

From Reusing Ideas to Reusing Code

We just discussed **Patterns**, which is how we reuse abstract knowledge and design solutions. But what if we want to reuse actual, compiled code?

Most modern software is constructed by reusing existing components or systems rather than writing everything from scratch.

The Reuse Landscape

Levels of Reuse

- **Abstraction Level:** Reusing knowledge (Design/Architectural Patterns).
- **Object/Function Level:** Reusing libraries (e.g., Python's `math` or `datetime`).
- **Component Level:** Reusing frameworks (e.g., React.js for UI, Express.js for routing).
- **System Level:** Reusing entire applications (e.g., integrating MySQL or a Virtual Docker container).

Benefits & Risks of Software Reuse

Benefits	Problems
Accelerated Development: Bringing a system to market early is often more important than overall development costs.	Lack of Trust / Not-Invented-Here: Developers often prefer to rewrite components because they believe they can improve on them.
Increased Dependability: Reused software that has been tried and tested should be more dependable than new software.	Maintenance & Dependencies: If a reused component breaks or loses support, your system breaks.

Real-World Example: In 2016, a developer deleted an 11-line open-source component called `left-pad` from the NPM registry. Because thousands of other components depended on it, large parts of the global internet (including React and Node) temporarily broke!

Open-Source Development

Today, the primary vehicle for Software Reuse is **Open-Source Development**.

- **What is it?** An approach where the source code of a software system is published, and volunteers are invited to participate in the development process.
- **Roots:** Started by the Free Software Foundation, advocating that source code should not be proprietary.
- **Scale:** Extended by the Internet to recruit massive populations of volunteer developers and users.

Famous Real-World Examples: The Linux operating system, Apache web server, and MySQL database management system are massively successful open-source products.

Open-Source Licensing (The LEPSI Connection)

Just because source code is freely available does **not** mean you can do whatever you want with it. The developer legally owns the code and places restrictions via licenses.

Permissive Licenses (e.g., MIT, Apache 2.0)

Allow you to license your project however you want, even in closed-source, proprietary, for-profit products.

Copyleft Licenses (e.g., GPLv2, GPLv3)

"Strong" copyleft. Including any library with these licenses requires you to choose an identical or compatible license for your entire project.

Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns
- 3 Design Patterns
- 4 Software Reuse & Open Source
- 5 Component-Based Software Engineering**
- 6 Next Steps

Introduction to CBSE

We have seen how to reuse abstract knowledge (Patterns) and how the industry shares reusable code (Open Source). But how do we architect systems to ensure this reused code integrates flawlessly?

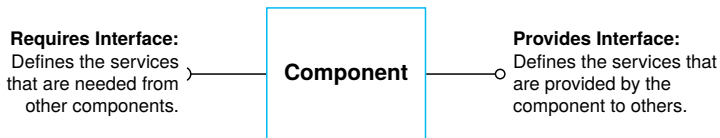
Component-Based Software Engineering (CBSE)

An approach to software development that relies on the reuse of entities called 'software components'.

- It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific.
- Components are more abstract than object classes and act as stand-alone service providers.
- They can exist as independent, executable entities.

CBSE Interfaces (The General Form)

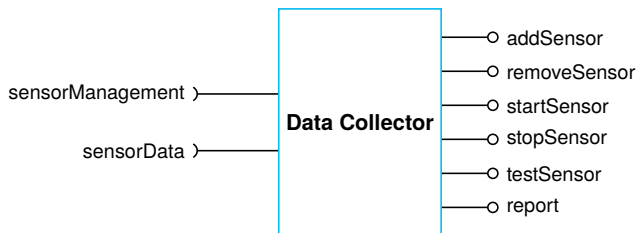
The services offered by a component are made available through an interface, and all interactions take place through that interface. We model this using UML "ball and socket" notation.



*Note: The 'requires' interface does not compromise independence, because it doesn't define **how** the external service is implemented, only that it is needed.*

CBSE Interfaces (A Concrete Example)

Let's look at how this general form applies to a specific subsystem: a **Data Collector** component.



The Data Collector provides an API to add and start sensors, but it explicitly requires external data and management subsystems to function.

Modern CBSE: Services & Microservices

Historically, CBSE relied on strict component models like Sun's Enterprise Java Beans (EJB), Microsoft's COM+/.NET, and CORBA.

The Problem

In practice, these standards hindered the uptake of CBSE because it was often impossible for components developed using different approaches to work together.

The Modern Solution: In response, the notion of **component as a service** was developed.

- Service-oriented components are stand-alone entities accessed over a network (e.g., RESTful APIs).
- Instead of language-specific standards, the industry adopted **Microservices** deployed in **Containerised units** (e.g. using Docker).
- You need only to reference the component via an endpoint, rather than importing its entire source code.

CBSE Processes

CBSE processes are software processes that take into account the possibilities of reuse and the different activities involved.

This splits the engineering workflow into two distinct approaches:

1 Development FOR Reuse:

- Concerned with developing components or services that will be reused in other applications.
- It usually involves generalizing existing components so they are abstract enough to be helpful to others.

2 Development WITH Reuse:

- The process of developing new applications using existing components and services.
- Instead of writing from scratch, engineers act as integrators, wiring together databases, API services, and UI frameworks.

Agenda

- 1 Recap: Week 4
- 2 Architectural Patterns
- 3 Design Patterns
- 4 Software Reuse & Open Source
- 5 Component-Based Software Engineering
- 6 Next Steps**

Reading & Resources

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapters 6, 7, 16, and 17.
AND
the original "Gang of Four" book: Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*.
- **refactoring.guru/design-patterns:** An interactive resource for learning Design Patterns with code examples in Python, Java, C#, and TS.
- **ChooseALicense.com:** GitHub's guide to understanding open-source licenses.

Next Week: Human-Computer Interaction

Any Questions?

`fdelduchetto@lincoln.ac.uk`